II2202 · FINAL PRESENTATION

Synchronization Latency for Databases

PowerSync versus ElectricSQL

Anita Guðrún Finnsdóttir · Olivia Hultman

KTH Royal Institute of Technology · 23 October 2026

PROBLEM STATEMENT

No one has measured how fast sync engines sync*

Context

Local-first apps keep the primary copy of data on the device. A sync engine keeps the replicas consistent, which enables collaboration and offline work.

Gap

Existing comparisons rely on documentation and developer surveys. Vendor figures are measured differently and cannot be compared.

Goal

Measure **synchronization latency**: the time from a write in Postgres until it is readable on a client, for PowerSync and ElectricSQL.

RESEARCH QUESTIONS

RQ1

How do PowerSync and ElectricSQL compare in synchronization latency?

RQ2

How is their synchronization latency affected when network latency increases?

DELIMITATIONS

2 engines · 1 varied parameter (downstream network latency) · 1 metric · one client, self-hosted on one machine

* To our knowledge, no empirical, independent, fair study of sync engines has been done before

BACKGROUND AND RELATED WORK

Local-first software depends on sync engines

Local-first software

The primary copy of data lives on the device. Kleppmann et al. define Seven Ideals, e.g. fast, multi-device, offline and collaborative [1].

Sync engines

Apply changes to every replica and make them converge. PowerSync and ElectricSQL are relational partial-sync engines [2].

Inconsistency window

Time from a committed write until a reader can see it. Benchmarked for cloud storage by Bermbach and Tai [3], not for sync engines.

Related work: Shiferaw [2] compared six engines using documentation and a survey of 12 developers, and named controlled benchmarking as the missing next step.

The two engines take different paths to the client

PowerSync

Postgres
logical replication stream

→

PowerSync Service
writes to bucket storage, then streams

→

Client SDK
writes the row to SQLite, then notifies

→

App reads
from local SQLite

ElectricSQL

Postgres
logical replication stream

→

Electric
serves shapes over an HTTP API

→

Client
receives the shape log; developer applies it

→

App reads
from local SQLite (our choice)

Why it matters: PowerSync decides when a row is readable; ElectricSQL leaves it to the developer, so we also apply its shape log to SQLite.

RESEARCH METHOD

A controlled testbed, self-hosted on one machine

Orchestrator
Node.js; inserts a row and logs t0

→

Postgres 16
wal_level = logical

→

Sync engine
PowerSync or Electric, in Docker

→

Toxiproxy
adds 0, 25, 50 or 100 ms downstream

→

Client
Node.js + SQLite; logs t1

t0 · just before INSERT | **notified_at** | **t1 · readable**

Delivery: Postgres → sync engine → client is notified | Local: row readable in SQLite

Synchronization latency = t1 – t0, the inconsistency window [3]

Linux Mint 22.3 · Intel Core Ultra 7 155H · 30 GiB RAM · Docker 29.2 · Node.js 25 · same table (id, payload, inserted_at) in Postgres and SQLite

RESEARCH METHOD

Identical workloads, 300 samples per condition

2 × 4

engines × added latency levels: 0, 25, 50 and 100 ms

300

samples per engine and level: 3 runs × 100 inserts

30

warm-up inserts per level, not measured

100 → 0

one run in reverse level order, to rule out time effects

ANALYSIS · PANDAS, SCIPY, NUMPY

RQ1 · engine comparison

Welch's t-test, Mann–Whitney U, bootstrap 95% CI of the mean difference

RQ2 · network latency

Linear regression per engine; slopes compared with a z-test and refitted per run

Outliers flagged with the 1.5 × IQR rule but kept: they are real synchronizations.

RESULTS · RQ1 · NO ADDED NETWORK LATENCY

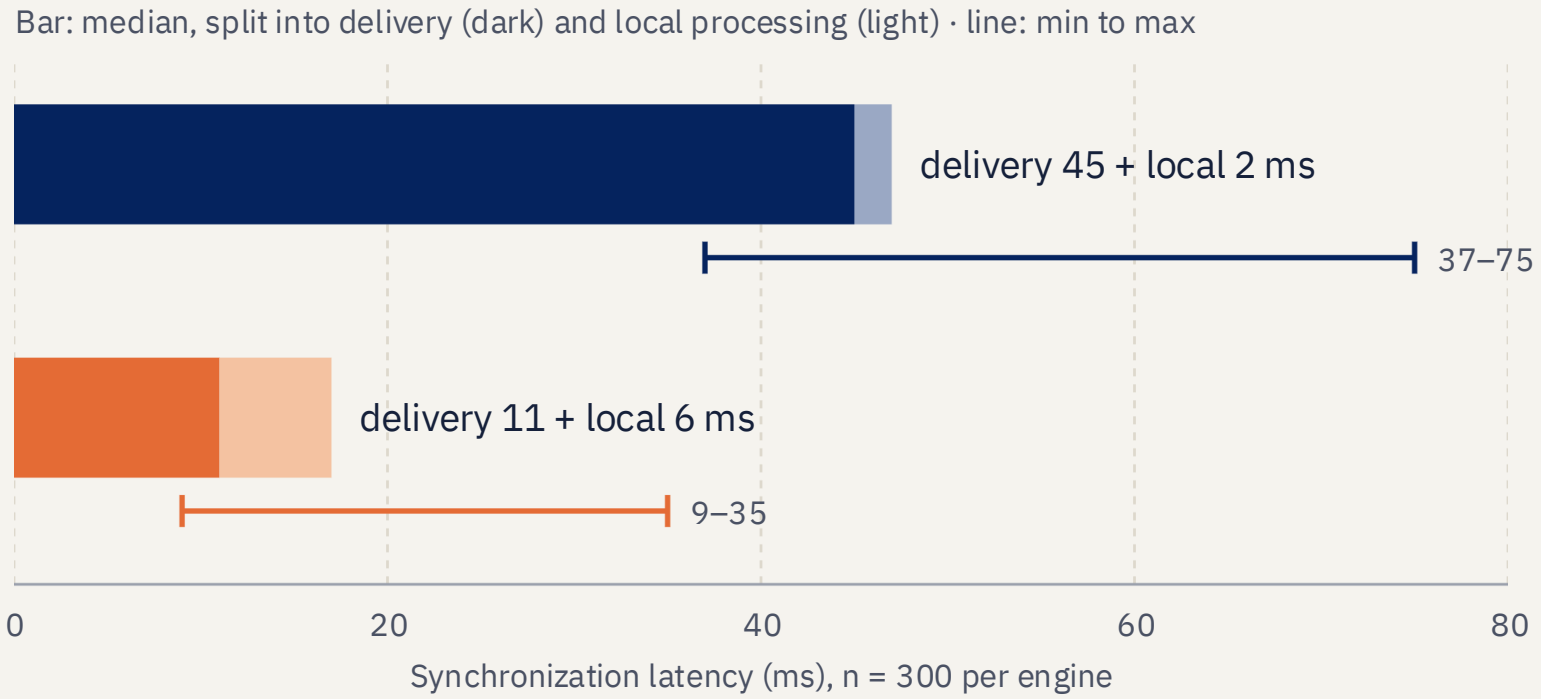
ElectricSQL synced 30 ms faster than PowerSync

PowerSync · median

47 ms

ElectricSQL · median

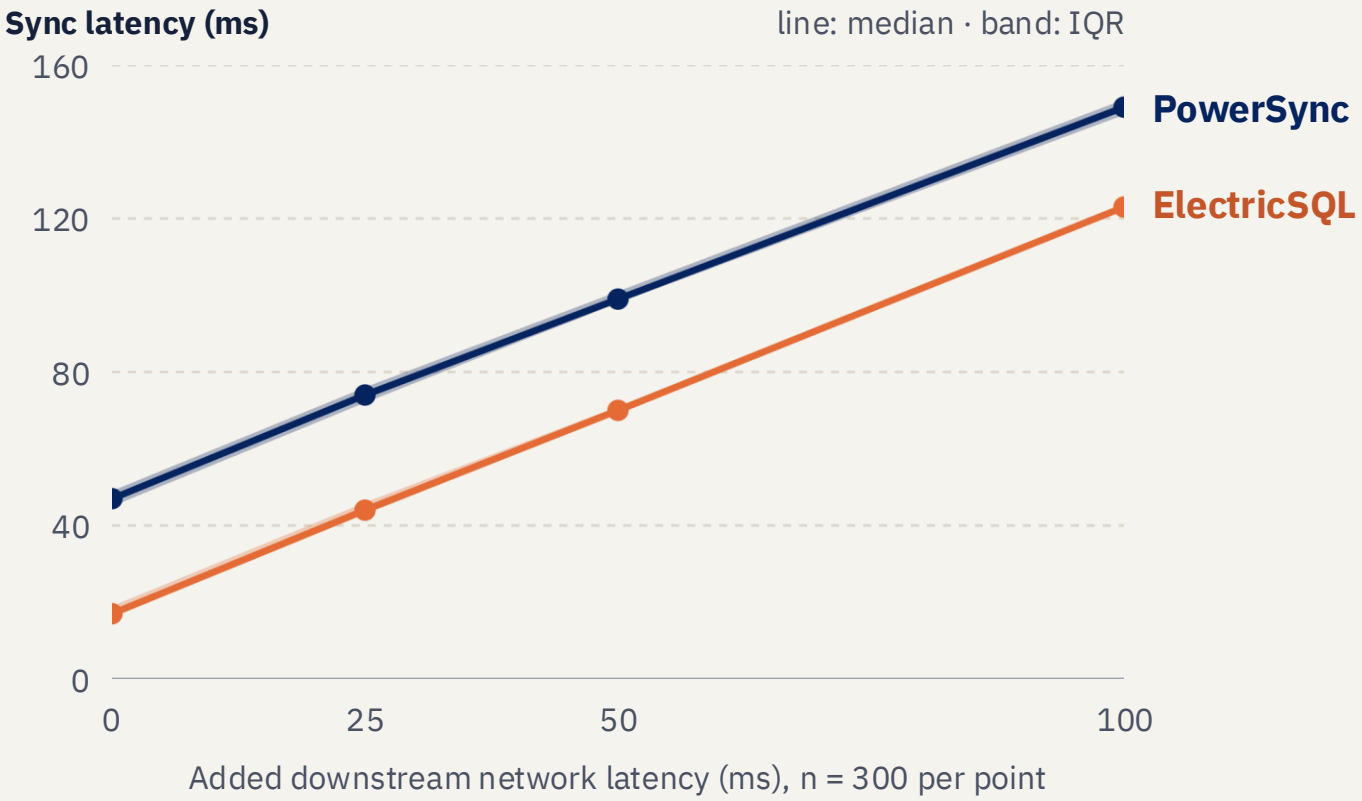
17 ms



Mean difference **29.9 ms** (95% CI 29.3–30.5) · Welch's t = 101.9, p < 0.001 · Mann–Whitney U = 90 000, the maximum: **every PowerSync sample was slower than every ElectricSQL sample**

RESULTS · RQ2 · ADDED NETWORK LATENCY

Added network delay passes almost straight through



PowerSync slope

1.021 ms/ms

95% CI 1.014–1.028 · R^2 = 0.986

ElectricSQL slope

1.053 ms/ms

95% CI 1.049–1.057 · R^2 = 0.995

Difference 0.032 ($z = 7.88$, $p < 0.001$): only **~3 ms extra per 100 ms** of delay. The gap narrows from 30 to 26 ms.

CONCLUSION AND FUTURE WORK

The engine matters most on fast networks

1. ElectricSQL synced faster: median **17 ms vs 47 ms** for PowerSync, with no overlap between the distributions.
2. The gap lies in **delivery**, before the client is notified; local processing took only 2–6 ms.
3. Network delay passed through **almost 1:1** for both engines (slopes 1.02 and 1.05).
4. The inconsistency-window method carries over to local-first sync engines.

Limits: one machine, one client, self-hosted, simple insert workload. Results hold for this setup only.

FUTURE WORK

A second client

Time from a write on client A until it is visible on client B, with three timestamps.

More network conditions

Jitter and packet loss, not only fixed latency.

Questions?

Anita Guðrún Finnsdóttir · Olivia Hultman

Code and data: github.com/OliviaHultman/powersync-vs-electricsql